

How to make a code plan

Handoff guide for a planning LLM and AI Builder

Version 0.3 planning contract | 24 September 2026

Give this guide and your idea to the LLM that will write your project plan. Its job is to produce one self-contained, executable plan for AI Builder. Save that resulting plan as Markdown, a text-based PDF, Word, or plain text and drop it into the app. Do not use this general guide itself as the project to build.

The plan must answer: what should work, which files implement it, what each task changes, how to test it, how to launch it, and what to do when a check fails. Small, complete, testable increments are more reliable than a large list of vague instructions.

What this can and cannot promise

A precise plan reduces ambiguity and repeated work. It cannot guarantee that a local model will never make mistakes, stall, misunderstand a requirement, or write weak tests. The application executes checks, limits retries, records progress, and saves checkpoints. A person must still try the finished software and review requirements that automated tests do not cover.

Choose the language for the job

- For a small utility and broad library ecosystem, consider Python. Prefer its standard library for the first build; external packages need separate setup.
- For native performance with stronger memory-safety guarantees, consider Rust when its compiler is installed. Safe Rust prevents many memory errors; unsafe code and foreign libraries can weaken that protection. [1, 2]
- For portable command-line programs and services, Go is another candidate when available. Its runtime manages memory and provides concurrency facilities. [3]
- For browser interfaces, choose JavaScript or TypeScript. TypeScript adds checking during development; it does not replace JavaScript's runtime. [4]
- Choose C or C++ when a low-level interface, existing native library, or measured performance need justifies the additional memory-management responsibility. C being fast does not make it automatically safer or easier to maintain.

Do not ask for a rewrite in C solely because C is low-level. State a measurable performance target first. Keep correctness, memory safety, dependency quality, and portability as separate requirements.

Planning defaults

Target one small useful result, one main language, standard libraries, an offline build, explicit file paths, and deterministic tests. Prefer 1-3 implementation tasks, each leaving runnable software or a testable library. Larger work should be split into separate project milestones.

Language profiles and actual readiness

These are 15 commonly used languages, not a definitive popularity ranking. A configured profile is not proof that every framework or dependency works. Check the app's current tool status before selecting a language.

Language	Current Mac readiness	Best initial scope
C	Real model build verified	Native CLI, reusable static library
C++	Compile/run smoke verified	Standard-library native CLI
Python	Real model build verified	Utilities, data transformations, tests
JavaScript	Runtime smoke verified	Node utilities, static websites
TypeScript	Needs tsc	Typed web/Node code compiled to JS
Rust	Needs rustc	Standard-library native utilities
Go	Needs go	Standard-library CLI or services
Java	Needs a JDK	JVM CLI with explicit assertion tests
C#	Needs .NET SDK	Prepared offline .NET console project
Swift	Compile/run smoke verified	Portable standard-library CLI
Kotlin	Needs Kotlin compiler and JDK	JVM CLI and self-contained test jar
Ruby	Runtime smoke verified	Standard-library scripts
PHP	Needs PHP CLI	CLI utilities with explicit test failures
Lua	Needs Lua	Small scripts and assertions
R	Needs Rscript	Base-R analysis and stopifnot tests

This is a snapshot of the development Mac, not a claim about your other machines. Missing tools are not installed automatically. For an unverified profile, begin with a tiny compile/run/test proof before a larger plan. C# currently needs an already-prepared offline build environment; do not promise automatic NuGet restoration.

Language-specific command patterns

- C/C++: compile all required sources and test executables, then run `./build/test_app`. Use clang or clang++; portable exports choose cc or c++ on the destination.
- Rust: `rustc --edition=2021 main.rs -o build/app`; `rustc --edition=2021 --test main.rs -o build/test_app`; `./build/test_app`. Cargo downloads are outside this prototype.
- Go: `go build -o build/app .`; `go test ./...` with local modules and no downloads.
- Java: `javac -d build Main.java TestMain.java`; `java -ea -cp build TestMain`.
- Swift: `swiftc Main.swift -o build/app`; compile and run a separate assertion-based test executable. Avoid Apple-only frameworks for Linux targets.
- Interpreted languages: invoke the runtime explicitly and make failed assertions return a nonzero exit code. TypeScript and Kotlin need their compile steps before runtime tests.

Commands above are separate commands, not one shell string. Never join them with semicolons, pipes, or `&&` in the submitted plan.

The execution contract

AI Builder is a macOS desktop app. It sends the plan to an LM Studio model on localhost, converts the plan to ordered tasks, and asks the model for complete file contents, check commands, and a launch command. The app writes allowed files, executes commands, feeds failures back for repair, and checkpoints passing tasks.

What the planner must assume

- The desktop importer accepts readable PDF, DOCX text, Markdown, and text. Prefer selectable text, normal headings, and code blocks. Scanned PDFs need OCR first; embedded images are not sufficient specifications.
- Plans must be at most 10 MB and 60,000 extracted characters. Current source context is limited to 100,000 characters. Keep the project small enough to fit with tests and imported source.
- The model sees the plan, current task, relevant supported text files, and failure feedback. Never rely on an earlier chat, an unattached reference, or a link the local model would have to fetch.
- Execution is sequential. Each task gets at most three generation/check attempts. A failed compile stops subsequent checks. Native test binaries must be compiled in that check sequence before they run.
- Commands run without a shell, with no network access, within the chosen project workspace. Specify token arrays or simple separate commands. No wildcard expansion, shell redirects, package installs, or downloads.
- Normal check commands have a 60-second limit; running the completed program in-app has a 300-second limit. Prefer fast tests. Interactive input belongs in Run in Terminal, not automated checks.
- The app creates parent directories for declared build outputs. Keep compiled artifacts in build/. Do not put source files in build/, dist/, target/, bin/, or obj/ because exports exclude those generated directories.
- A stopped build preserves files and passing task records. Resume skips saved passing tasks and retries unfinished work. The final check pass is still required.

Application-owned files

Do not ask the coding agent to create or overwrite project.py, project.json, START.command, HOW_TO_RUN.md, BUILD_REPORT.md, .gitignore, PROJECT_PLAN.md, .git/, or .builder* state. The app owns those files. Ask it to write the project's README.md and normal source/tests instead.

Libraries and external assets

List every dependency with version expectation, purpose, license, source location, and setup owner. Distinguish standard libraries, imported local source, already-installed libraries, and unavailable packages. C source import copies a small .c/.h tree and license files into vendor/. Installed C libraries use explicit flags or named pkg-config packages. Do not invent downloaded assets or assume a package manager will run automatically.

If a prerequisite is missing, mark it as a blocker before implementation. Use a standard-library alternative only when the user accepts the functional tradeoff. Keep secrets, real accounts, private data, and deployment out of a first proof of concept.

Required contents of the project plan

Use these headings and write all essential details directly in the plan. They are a recommended planning contract; the desktop importer does not enforce every heading.

Project and user outcome

Give the name, one-sentence goal, intended user, and a concrete end-to-end example. Explain what the user will enter or click and what should happen. State the artifact type: CLI, library plus consumer demo, script, website, or document generator.

Scope and non-goals

Number requirements R1, R2, and so on. Give each requirement an observable result. List excluded features to prevent feature creep. Separate essential behavior from later improvements. Describe errors, empty input, limits, persistence, and how to exit or stop.

Language and target environment

Choose one primary language and a version/standard. Name macOS and Linux targets, CPU assumptions, required runtimes, and available tools. State whether native executables, runtime source, or browser files will be delivered. Identify Apple-only or Linux-only APIs explicitly.

Architecture and file map

List each source/test file and its responsibility. Define module boundaries and dependencies. Give exact public function names, argument types, return values, error behavior, and data formats. Explain state ownership and storage paths. Include one valid and one invalid input/output example per critical interface.

Dependencies and permissions

List allowed libraries and where they come from. Prefer local deterministic fixtures. Declare network requirements separately; the current build/test sandbox has no network. Do not include real credentials. Describe any manual installation prerequisite before the build.

Ordered tasks

Use stable IDs T1, T2, and so on. Each task names dependencies, files it may change, behavior to implement, checks, expected evidence, and a stop condition. Keep each task self-contained: code, tests, and verification belong together. Never leave a task consisting only of an empty header or placeholder UI.

Final acceptance and delivery

Map R1...Rn to automated tests or named manual checks. Include fresh build commands, regression tests, launch instructions, expected output, and Linux rebuild instructions. Require a README and document known limitations. Define completion as passing checks plus disclosed manual review items, not the model saying it is finished.

Make quality measurable

Replace “fast,” “secure,” “beautiful,” and “works well” with examples or thresholds. For instance: reject malformed input without crashing; preserve existing data on validation failure; complete a 1,000-record local test within a measured target; show an empty-state message; provide keyboard-accessible controls. Avoid performance claims without measuring on the target machine.

Task design that reduces repeated work

Write tasks in dependency order. Start with one thin working path through the software, then add behavior without changing previously agreed interfaces. Keep a single source of truth for requirements and data formats.

Copyable task format

T1 - Implement the conversion library and its first consumer
Depends on: none
Requirements: R1, R2
Files: include/temperature.h, src/temperature.c, main.c,
tests/test_temperature.c, README.md
Interfaces: exact signatures and units listed in Architecture
Work:
1. Implement the public interface without placeholders.
2. Add deterministic normal, boundary, and invalid-input tests.
3. Compile the library, consumer, and tests in that order.
4. Run tests and compare actual output with the expectations.
Checks: explicit argument arrays; no shell operations
Pass evidence: all exit codes 0; assertions executed;
expected output documented below
Preserve: do not rename the public interface or weaken tests
On failure: inspect the first failing diagnostic; fix the
relevant code; rebuild and rerun the same checks
Stop: if the environment or requirements are incompatible,
report the exact blocker instead of inventing a workaround

Rules for the planning LLM

1. Resolve decisions before splitting work. If necessary, ask only the questions that materially change architecture. Otherwise label reasonable assumptions and keep them consistent.
 2. Use concrete names throughout. Do not alternate between different module names, output folders, interfaces, or launch commands.
 3. Avoid placeholders such as “implement the remaining logic,” “add appropriate tests,” or “make it production ready.” Specify the behavior and evidence.
 4. Keep tests deterministic and offline. Use fixed fixtures, temporary project-local data, bounded timeouts, and reproducible seeds when randomness is essential.
 5. Write negative and boundary cases, not just a happy path. For a library, test the public interface through a separate consumer.
 6. Do not remove or weaken failing tests to manufacture success. Update tests only when an explicit requirement change makes them obsolete, and explain the change.
 7. Separate build commands from behavioral checks. A successful compiler run is not evidence that the program produces correct results.
 8. Keep changes relevant to the active task. Do not repeatedly rewrite the whole project or introduce a new framework halfway through.
 9. Include all inputs needed to reproduce a failure. The model should receive the failing command, exit code, expected output, and actual diagnostic.
 10. Finish with a complete regression run and a user-facing launch path. If manual visual checks remain, name them explicitly.
- The current application supplies bounded retries and checkpoints. These planning rules guide model behavior; not every rule is mechanically enforced, and human review still matters.

Reusable request for the planning LLM

Copy the following request and attach this guide. Replace the bracketed fields with your idea and environment details. The result should be the project-specific plan, not implementation code and not a discussion of how planning works.

Create one self-contained project plan for AI Builder using the attached guide "how to make a code plan".

My idea: [describe the intended software and user outcome]
Preferred language: [language, or recommend one with reasons]
Target computers: [macOS/Linux and known CPU architectures]
Available compilers/runtimes: [copy the app's tool status]
Allowed dependencies: [standard library/local source/installed]
Important constraints: [offline, performance, data, UI, budget]

First check that the project fits the builder's actual scope. Do not assume unsupported packages, tools, GUI frameworks, downloads, or a network-enabled test environment. Ask only essential missing questions; otherwise label your assumptions.

Return a ready-to-save Markdown plan with these sections:

1. Project name, goal, user outcome, and one complete example.
2. Numbered requirements R1...Rn and explicit non-goals.
3. Language, runtime versions, targets, and prerequisites.
4. Architecture, file map, exact interfaces, and data formats.
5. Dependencies, licenses, permissions, and input fixtures.
6. Ordered tasks T1...Tn, normally 1-3 complete increments.
7. Final acceptance checks, manual checks, and limitations.
8. How to build, test, run, and move the source to Linux.

For every task include dependencies, requirement IDs, exact files, implementation steps, test commands, expected results, preserved behavior, failure recovery, and a stop condition. Every task must leave useful behavior with executable tests.

Use explicit command argument arrays. Compile native tests before running them. Do not use shell chaining or wildcards. Do not alter application-owned files. Keep source outside build-output folders. Do not claim external assets exist.

Review your plan for missing steps, contradictory interfaces, untestable criteria, unavailable prerequisites, and scope creep. Fix those problems before returning the plan. Do not promise perfect model behavior or claim software has been tested yet.

Before handing the result to the local model

Read the plan once. Confirm the language and prerequisites are actually available, the tasks are small, the output is useful, and the tests would catch an incorrect implementation. Save it as a separate file with a meaningful project name. In AI Builder, choose a project name and destination, select the matching language and local model, then import that plan.

Complete compact example

Project

Name: Temperature Library. Goal: deliver a reusable C11 static library and a command-line demo that work from the same source on macOS and Linux. No external dependencies or network access.

Requirements and scope

R1: Convert Celsius to Fahrenheit and Fahrenheit to Celsius using double precision. R2: Expose a documented header for another C program to use. R3: A demo prints 68.0 for 20 Celsius. R4: Tests cover freezing, boiling, and negative temperatures in both directions. Exclude interactive UI, files, localization, and extreme/non-finite floating-point inputs in this milestone.

Architecture and interfaces

- include/temperature.h: declares `double celsius_to_fahrenheit(double c)` and `double fahrenheit_to_celsius(double f)`, with include guards.
- src/temperature.c: implements $f = c * 9.0 / 5.0 + 32.0$ and $c = (f - 32.0) * 5.0 / 9.0$. No global mutable state or allocations.
- main.c: calls the library for 20.0 Celsius and prints the result with one decimal place.
- tests/test_temperature.c: a separate main function tests the public interface with assert and a floating-point tolerance of 1e-9. Include math.h if using fabs.
- README.md: describes prerequisites, interface, build, test, demo, and portability.

T1 Implement library consumer and tests

Dependencies: none. Requirements: R1-R4. Create the five files above. Compile an object, archive it, link the demo and tests, then run both. Keep all generated outputs in build/. Do not modify the app-owned manifest or runner.

```
["cc", "-std=c11", "-Wall", "-Wextra", "-Iinclude",  
  "-c", "src/temperature.c", "-o", "build/temperature.o"]  
["ar", "rcs", "build/libtemperature.a", "build/temperature.o"]  
["cc", "-std=c11", "-Wall", "-Wextra", "-Iinclude",  
  "main.c", "build/libtemperature.a", "-o", "build/app"]  
["cc", "-std=c11", "-Wall", "-Wextra", "-Iinclude",  
  "tests/test_temperature.c", "build/libtemperature.a",  
  "-o", "build/test_temperature"]  
["./build/test_temperature"]  
["./build/app"]
```

Pass evidence: every command exits 0; all assertions execute; the demo prints 68.0. On failure, repair the first diagnostic and rerun the full sequence. Do not skip assertions, run an older binary, or change the interface. Stop if a compiler prerequisite is unavailable. Launch command: `["./build/app"]`.

Final review: rerun the sequence from source, verify README steps, and disclose that Linux still needs a build/test on the Linux machine. The app-generated project.py helper can then build, test, and run the exported source.

Testing delivery and portability

How the user tests a result

1. Open the exact output folder shown in AI Builder. The default parent is Documents / AI Builder Projects; a chosen destination overrides it.
2. Use Test project for a fresh build and the recorded checks. Read failures rather than treating a created file as success.
3. Use Run project for ordinary output, or Run in Terminal for interactive input. For static HTML, try the site in a browser and verify layout, navigation, and keyboard behavior.
4. Inspect README.md and BUILD_REPORT.md. Try the end-to-end example from the plan and at least one bad input.
5. Use Open in VS Code to edit source. After edits, rebuild and test before running or exporting again.

What the output is

C, C++, Rust, Go, and Swift commonly produce native binaries. A macOS binary is not a Windows .exe and normally cannot run directly on Linux. Java and Kotlin generally need a compatible JVM. C# usually needs a compatible .NET runtime unless explicitly packaged otherwise. Python, Ruby, PHP, Lua, R, and Node programs need their runtimes. Websites are browser assets. The selected plan determines the artifact; the builder does not automatically create installers for every language.

Move it to another computer

Use Export source ZIP after successful checks, or put the reviewed source in a separate GitHub repository and clone it on the second computer. Both work; GitHub is optional. Do not upload generated binaries as a substitute for a portable build recipe.

In the transferred project folder, run:

```
python3 project.py build
python3 project.py test
python3 project.py run
```

Install the necessary runtime/compiler and approved dependencies on the destination first. Open the same folder in VS Code with File > Open Folder. Source exports omit build outputs, internal state, obvious secrets, and Git history; review the archive before sharing it. The portable helper uses normal user permissions, not the builder sandbox.

Final planning review checklist

- Every requirement has a test or an explicit manual review step.
- Interfaces, units, filenames, task order, and commands agree.
- Runtime/compiler availability is confirmed; dependencies are named and licensed.
- Every task includes implementation plus tests; no placeholder-only task remains.
- Negative and boundary cases are specified; no tests require private data or network access.
- The launch command and expected user-visible result are unambiguous.
- Existing behavior must be preserved during changes; failed builds cannot use stale output.
- Portability assumptions and untested platforms are disclosed.
- The plan fits the prototype's size, execution, and retry limits.

Sources for language guidance

[1] Rust performance and safety overview: <https://rust-lang.org/>

[2] Rust unsafe-code boundaries: <https://doc.rust-lang.org/reference/unsafety.html>

[3] Go language guidance: https://go.dev/doc/effective_go

[4] TypeScript runtime relationship: <https://www.typescriptlang.org/docs/handbook/typescript-from-scratch>

[5] Python application and library ecosystem: <https://www.python.org/about/apps/>

The builder-specific contract is based on this repository's implementation and local checks, not on those language documentation pages. Update this guide when commands, limits, language profiles, or file ownership change.